ECB, CBC, and CFB Modes of Operation

34. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

Aim:

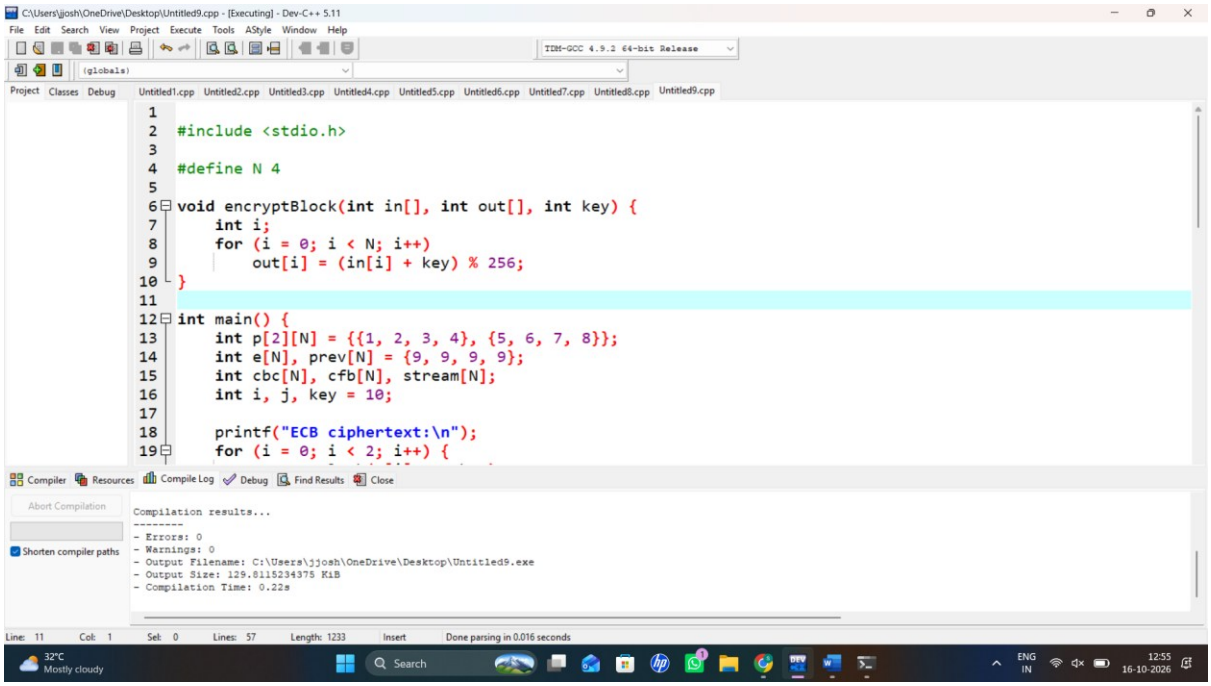
To implement the ECB, CBC, and CFB encryption modes in C and understand their behavior with complete data blocks.

Algorithm:

1. Read a plaintext consisting of complete blocks.
2. Select an encryption key and initialization vector (IV).
3. In ECB mode, encrypt each block independently.
4. In CBC mode, XOR each plaintext block with the previous ciphertext block before encryption.
5. In CFB mode, encrypt the previous ciphertext block or IV and XOR the result with the plaintext.
6. Display the ciphertext for each mode.
7. Explain the purpose of padding, including padding a message that is already block-aligned.

Program:

The following program uses a toy block transformation to demonstrate the modes. It is not a secure implementation of a real block cipher.



```c
#include <stdio.h>

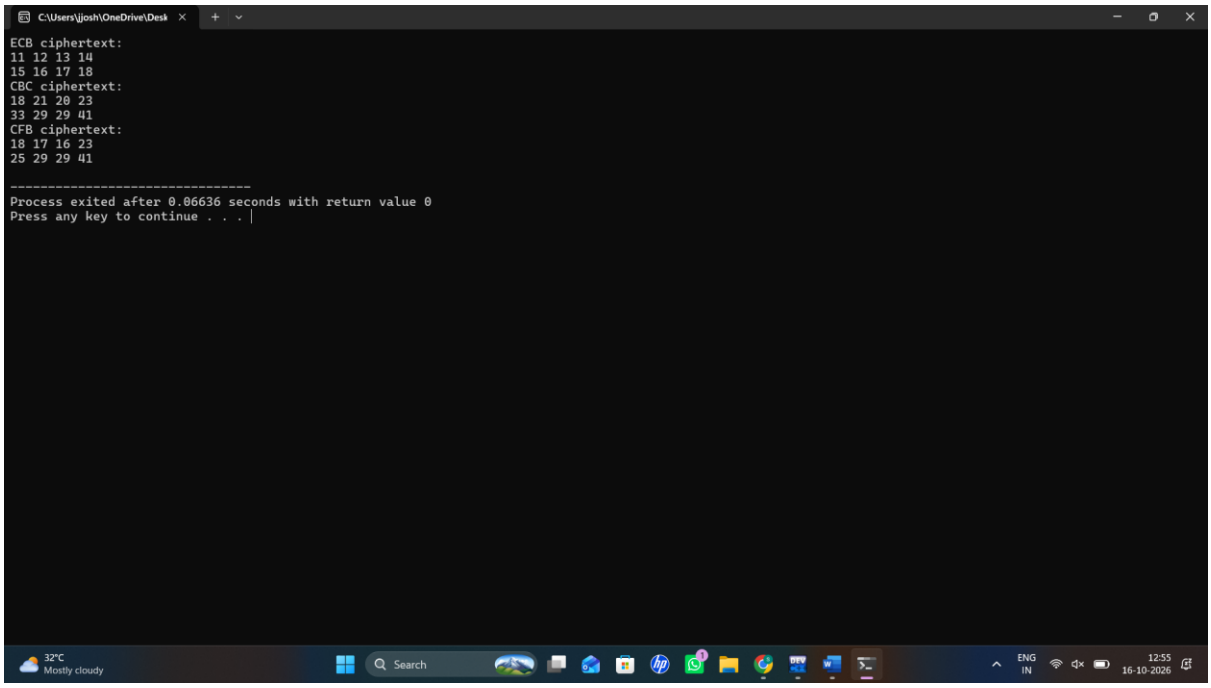
#define N 4

void encryptBlock(int in[], int out[], int key) {
    int i;
    for (i = 0; i < N; i++)
        out[i] = (in[i] + key) % 256;
}

int main() {
    int p[2][N] = {{1, 2, 3, 4}, {5, 6, 7, 8}};
    int e[N], prev[N] = {9, 9, 9, 9};
    int cbc[N], cfb[N], stream[N];
    int i, j, key = 10;

    printf("ECB ciphertext:\n");
    for (i = 0; i < 2; i++) {
```

Output:



```
ECB ciphertext:
11 12 13 14
15 16 17 18
CBC ciphertext:
18 21 20 23
33 29 29 41
CFB ciphertext:
18 17 16 23
25 29 29 41

--------------------------------
Process exited after 0.06636 seconds with return value 0
Press any key to continue . . .
```

Result:

ECB, CBC, and CFB modes were demonstrated using a toy block transformation. The modes process data differently: ECB encrypts blocks independently, CBC chains ciphertext blocks, and CFB uses encrypted feedback to generate a keystream.

Question: Why pad a message when padding is not needed?

Always adding a padding block can help distinguish message endings and make the padding convention unambiguous, including when the original message length is an exact multiple of the block size. For example, a 1 bit followed by zeros can be recognized and removed when the receiver knows the convention and the original data format. Padding alone does not provide authentication or prevent tampering.